Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Date of Performance: 29/08/2026 **Date of Submission: 29/08/2026**

Class: T1

Student Roll No: 2401036 **Student Name: Dhruv Hemdev**

EXPERIMENT – 5

AIM : To automate application deployment using Github/Jenkins.

LO MAPPED : LO3.

LO3 : Build and manage automated CI/CD pipelines for application development, testing, Packaging and deployment.

THEORY :

Continuous Integration and Continuous Deployment

CI/CD is a software development methodology that automates the process of taking source-code changes from development to a deployed application. It minimizes manual tasks and makes software delivery faster and more reliable.

Continuous Integration (CI):
Continuous Integration involves regularly adding developers' code changes to a common repository. Each change can be automatically built and tested, which helps detect errors early and keeps the project in a stable state.

Continuous Deployment (CD):

Continuous Deployment is the next stage of the CI process, where code that successfully passes the required checks is automatically delivered to the target environment. This avoids the need for manually transferring application files.

Jenkins:

Jenkins is an open-source automation tool used for performing tasks such as building, testing, and deploying applications. It can connect with source-code repositories like GitHub and execute the stages defined in a pipeline.

Jenkins Declarative Pipeline:

A Declarative Pipeline defines the CI/CD process in a structured manner using a Jenkinsfile. Different stages such as Build, Test, and Deploy can be specified in the file. Storing this configuration along with the application source code makes the process repeatable.

GitHub:

GitHub is a remote repository used to store and manage application source code. In this experiment, Jenkins obtains the application files and pipeline configuration from the GitHub repository.

CI/CD Flow:

Developer modifies code → GitHub Repository → Jenkins fetches code → Build → Test → Deploy → Updated Application

EXECUTION :

Step 1: Jenkins Initialization and Setup

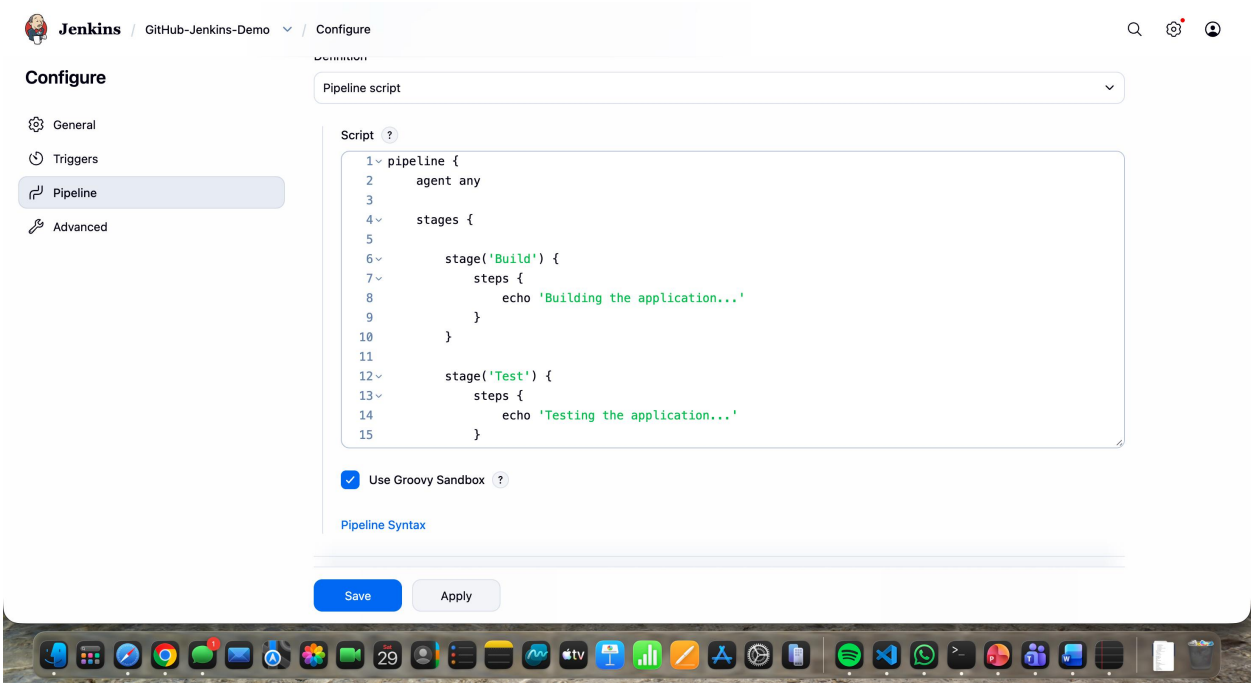
```
brew services start jenkins
==> Auto-updating Homebrew...
Adjust how often this is run with `$HOMEBREW_AUTO_UPDATE_SECS` or disable with
`$HOMEBREW_NO_AUTO_UPDATE=1`. Hide these hints with `$HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
jenkins 2.575 is already installed but outdated (so it will be upgraded).
==> Downloading bottle manifests
✔ Bottle Manifest jenkins (2.579)                  Downloaded  202.3KB/202.3KB
==> Would install 1 formula:
jenkins
==> Would upgrade 4 dependencies for jenkins:
fontconfig
harfbuzz
little-cms2
openjdk@21
==> Do you want to proceed with the installation? [y/n]
==> Fetching downloads for: jenkins
✔ Bottle fontconfig (2.18.3)                       Downloaded  451.6KB/451.6KB
✔ Bottle Manifest harfbuzz (14.4.0)                Downloaded  173.9KB/173.9KB
✔ Bottle Manifest little-cms2 (2.19.1)             Downloaded   67.1KB/ 67.1KB
✔ Bottle Manifest openjdk@21 (21.0.12.1)           Downloaded  233.0KB/233.0KB
✔ Bottle little-cms2 (2.19.1)                      Downloaded  432.8KB/432.8KB
✔ Bottle harfbuzz (14.4.0)                         Downloaded    3.7MB/  3.7MB
✔ Bottle openjdk@21 (21.0.12.1)                    Downloaded  200.5MB/200.5MB
✔ Bottle jenkins (2.579)                           Downloaded   63.4MB/ 63.4MB
==> Upgrading jenkins
  2.575 -> 2.579
==> Installing dependencies for jenkins: fontconfig ↑, harfbuzz ↑, little-cms2 ↑ and openjdk@21 ↑
==> Upgrading jenkins dependency: fontconfig
==> Pouring fontconfig--2.18.3.arm64_tahoe.bottle.tar.gz
🍺  /opt/homebrew/Cellar/fontconfig/2.18.3: 102 files, 1.9MB
==> Upgrading jenkins dependency: harfbuzz
==> Pouring harfbuzz--14.4.0.arm64_tahoe.bottle.tar.gz
🍺  /opt/homebrew/Cellar/harfbuzz/14.4.0: 96 files, 13.5MB
==> Upgrading jenkins dependency: little-cms2
==> Pouring little-cms2--2.19.1.arm64_tahoe.bottle.tar.gz
🍺  /opt/homebrew/Cellar/little-cms2/2.19.1: 23 files, 1.4MB
==> Upgrading jenkins dependency: openjdk@21
==> Pouring openjdk@21--21.0.12.1.arm64_tahoe.bottle.tar.gz
🍺  /opt/homebrew/Cellar/openjdk@21/21.0.12.1: 599 files, 346.7MB
==> Installing jenkins
==> Pouring jenkins--2.579.all.bottle.tar.gz
🍺  /opt/homebrew/Cellar/jenkins/2.579: 9 files, 65.5MB
==> Cleanup
==> jenkins
Removing: /opt/homebrew/Cellar/jenkins/2.575... (9 files, 86.0MB)
Removing: /Users/dhruv/Library/Caches/Homebrew/jenkins--2.575... (83.9MB)
```

```
==> jenkins
Removing: /opt/homebrew/Cellar/jenkins/2.575... (9 files, 86.0MB)
Removing: /Users/dhruv/Library/Caches/Homebrew/jenkins--2.575... (83.9MB)
Removing: /Users/dhruv/Library/Caches/Homebrew/jenkins_bottle_manifest--2.575... (202.1KB)
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
==> Caveats
==> jenkins
Note: When using launchctl the port will be 8080.

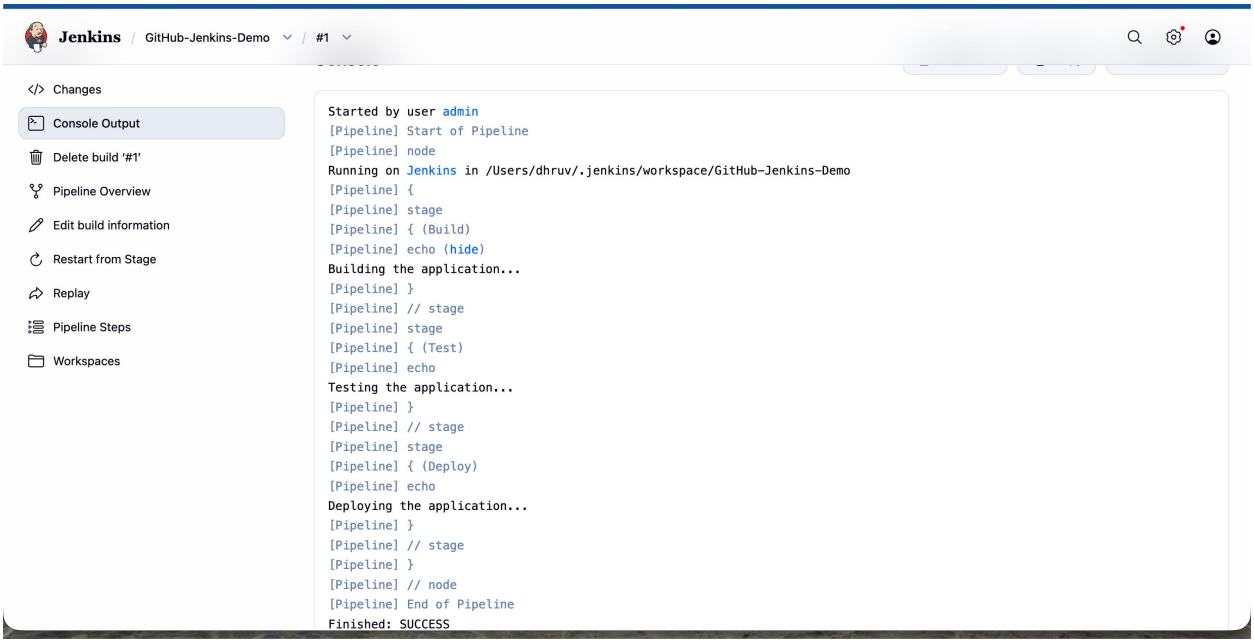
To start jenkins now and restart at login:
  brew services start jenkins
Or, if you don't want/need a background service you can just run:
  /opt/homebrew/opt/jenkins/bin/jenkins --httpListenAddress\=127.0.0.1 --httpPort\=8080
==> Successfully started `jenkins` (label: homebrew.mxcl.jenkins)
dhruv@Dhruvs-MacBook-Air ~ %
```

Step 2: Jenkins Pipeline Creation and Initial Validation

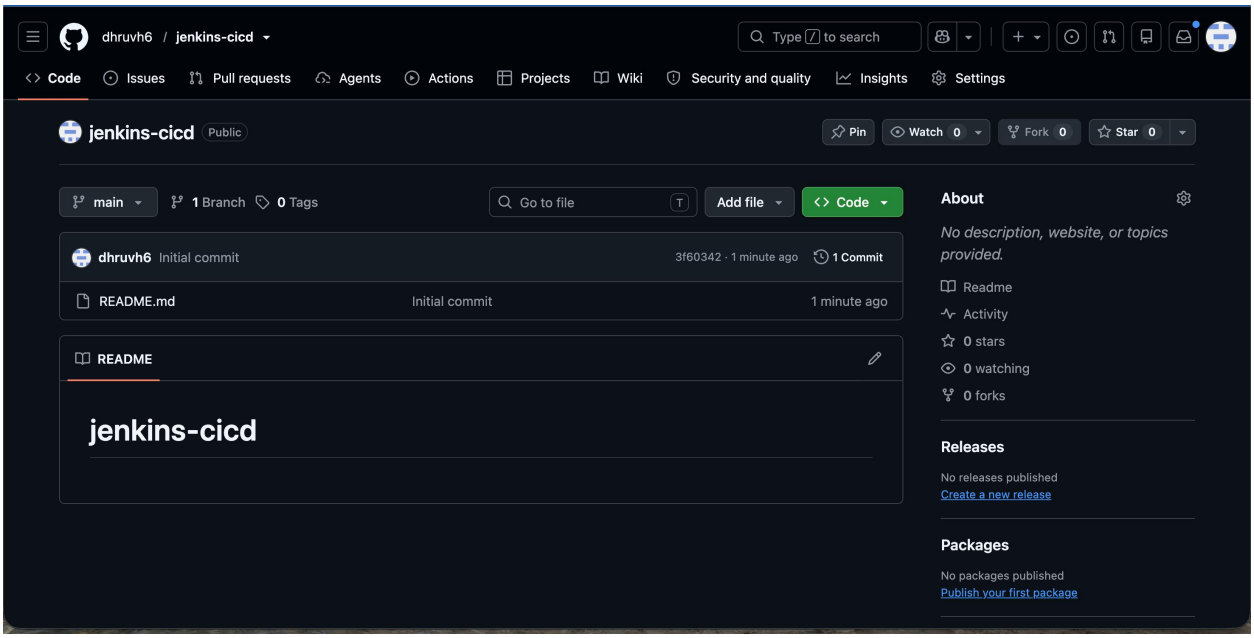
[SCREENSHOT MISSING — Jenkins dashboard — 'Welcome to Jenkins!' / job list]

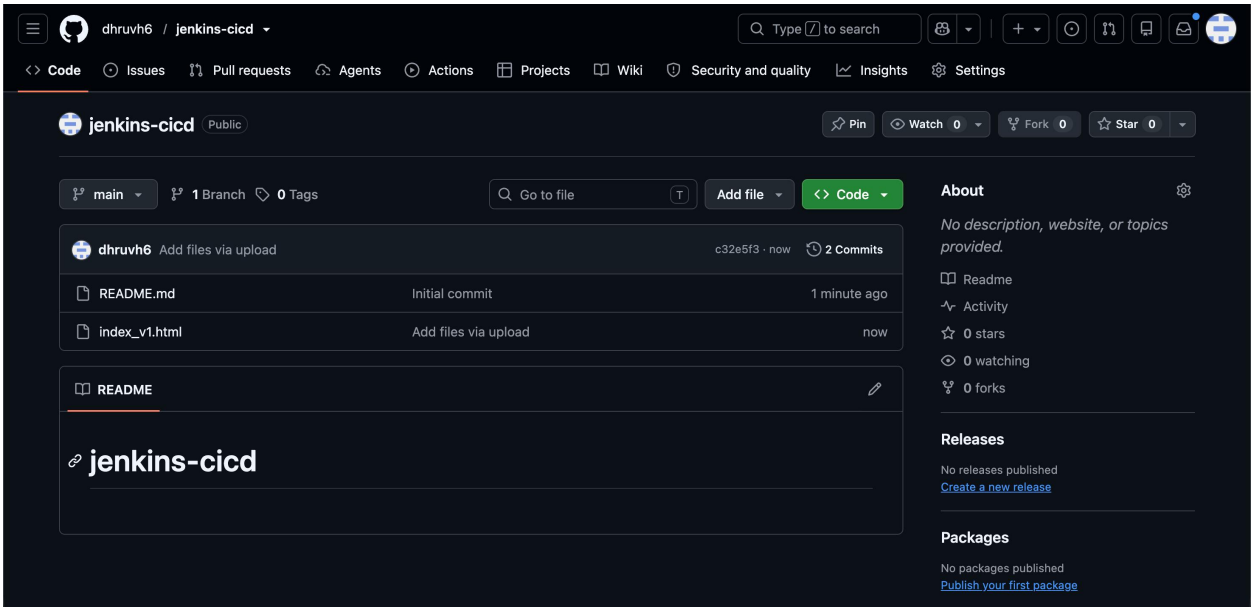


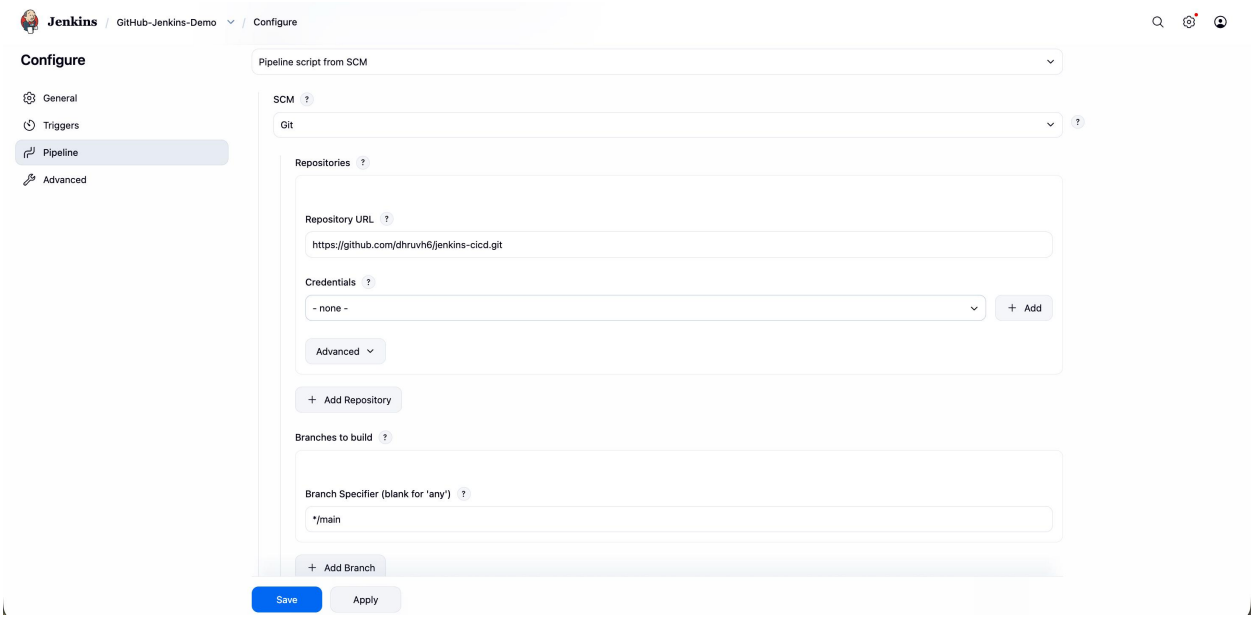
Step 3: Initial Pipeline Execution

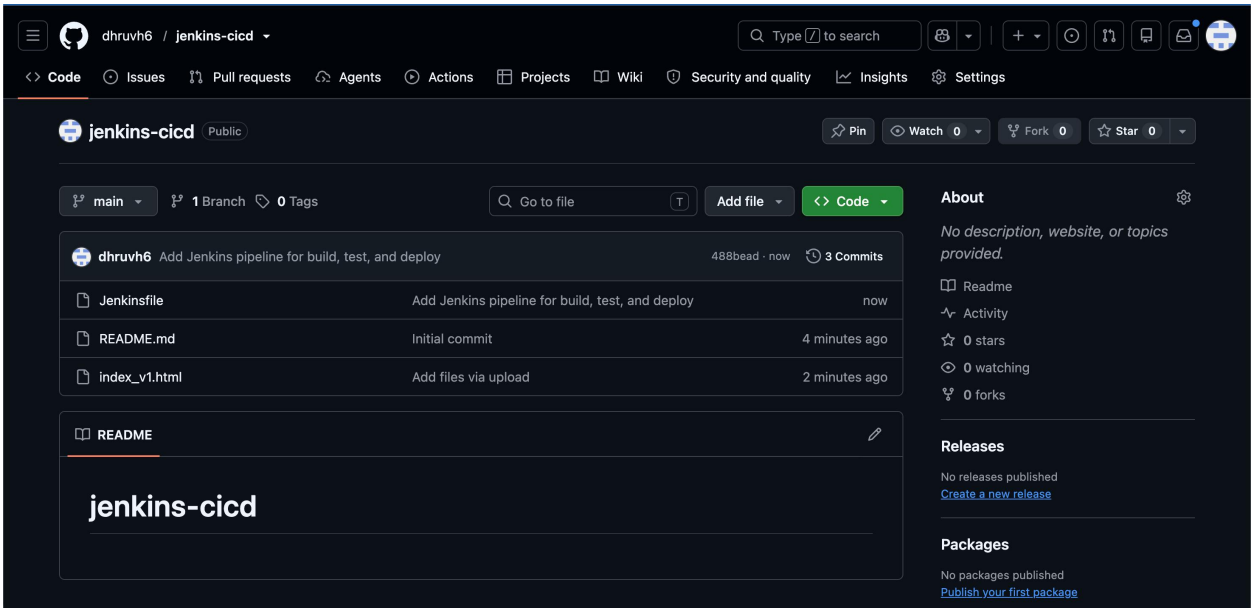


Step 4: GitHub Repository and Jenkinsfile Configuration

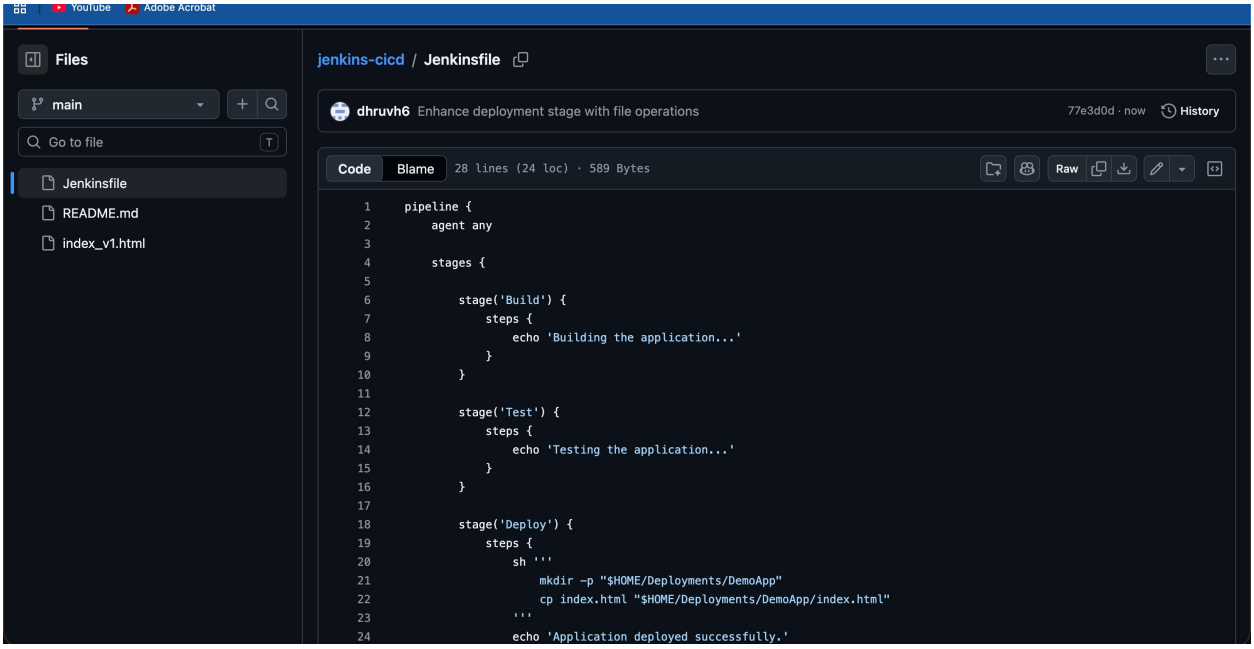



dhruvh6 / jenkins-cicd
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
jenkins-cicd
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
T
Add file
Code
dhruvh6 Add files via upload
c32e5f3 · now
2 Commits
README.md
Initial commit
1 minute ago
index_v1.html
Add files via upload
now
README
jenkins-cicd
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package


Jenkins
GitHub-Jenkins-Demo
Configure
Configure
General
Triggers
Pipeline
Advanced
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/dhruvh6/jenkins-cicd.git
Credentials
- none -
Add
Advanced
Add Repository
Branches to build
Branch Specifier (blank for 'any')
*/main
Add Branch
Save
Apply



Step 5: GitHub Checkout and Automated Deployment



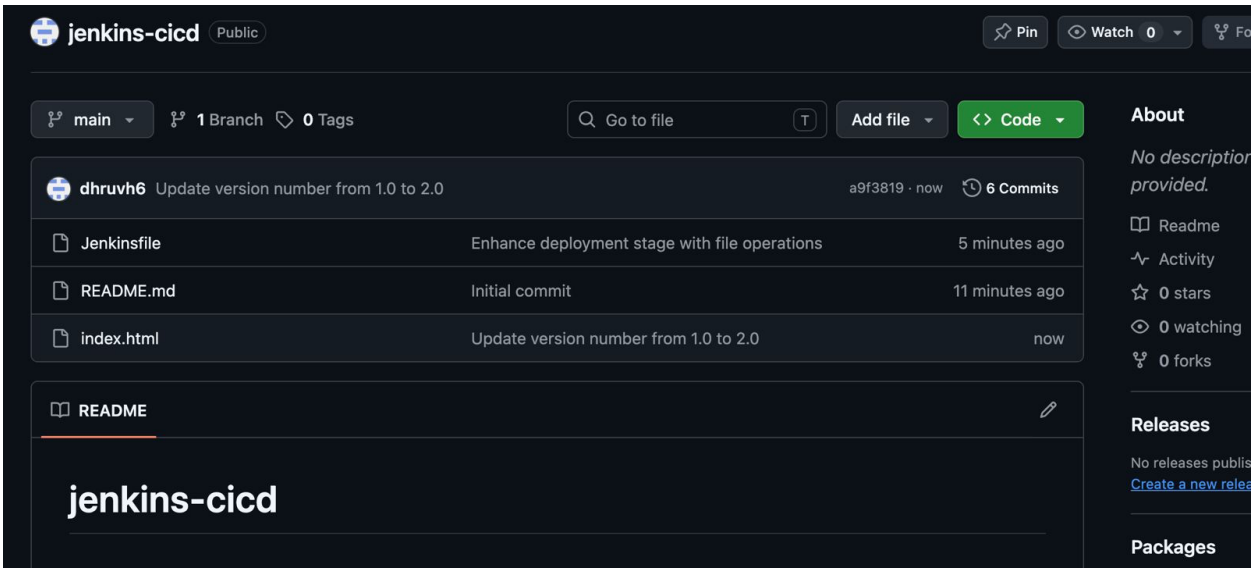
```
Started by user admin
Obtained Jenkinsfile from git https://github.com/dhruvh6/jenkins-cicd.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /Users/dhruv/.jenkins/workspace/GitHub-Jenkins-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
 > git rev-parse --resolve-git-dir /Users/dhruv/.jenkins/workspace/GitHub-Jenkins-Demo/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/dhruvh6/jenkins-cicd.git # timeout=10
Fetching upstream changes from https://github.com/dhruvh6/jenkins-cicd.git
 > git --version # timeout=10
 > git --version # 'git version 2.39.5 (Apple Git-154)'
```

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] sh
+ mkdir -p /Users/dhruv/Deployments/DemoApp
+ cp index.html /Users/dhruv/Deployments/DemoApp/index.html
[Pipeline] echo
Application deployed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

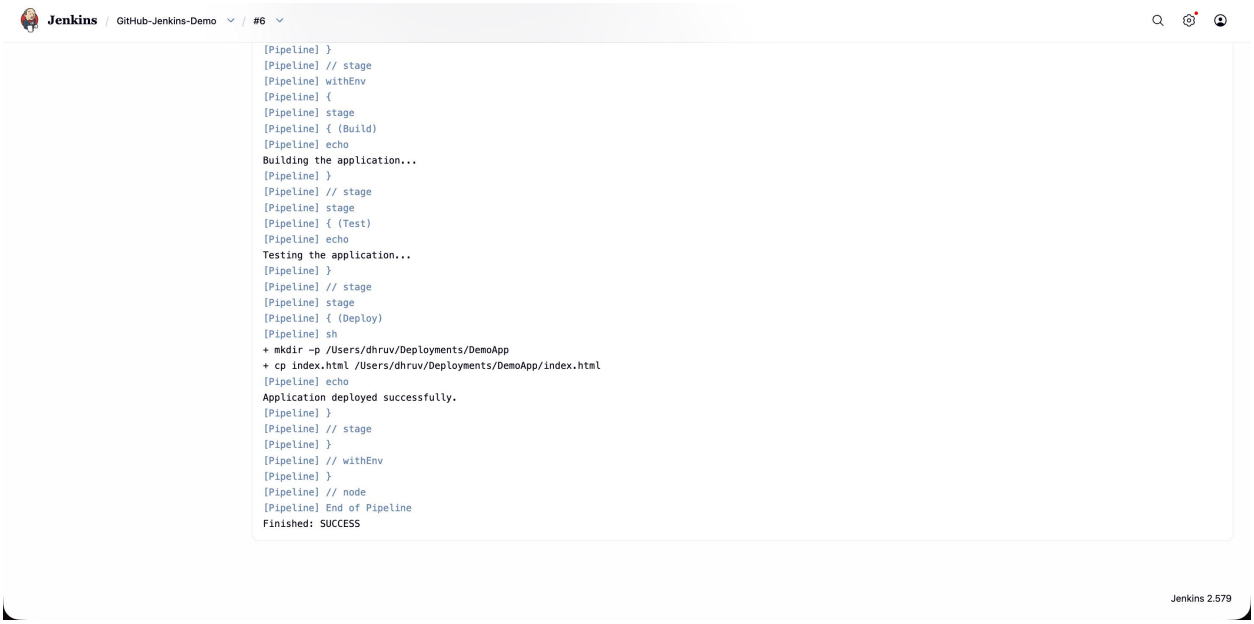
Step 6: Initial Application Deployment



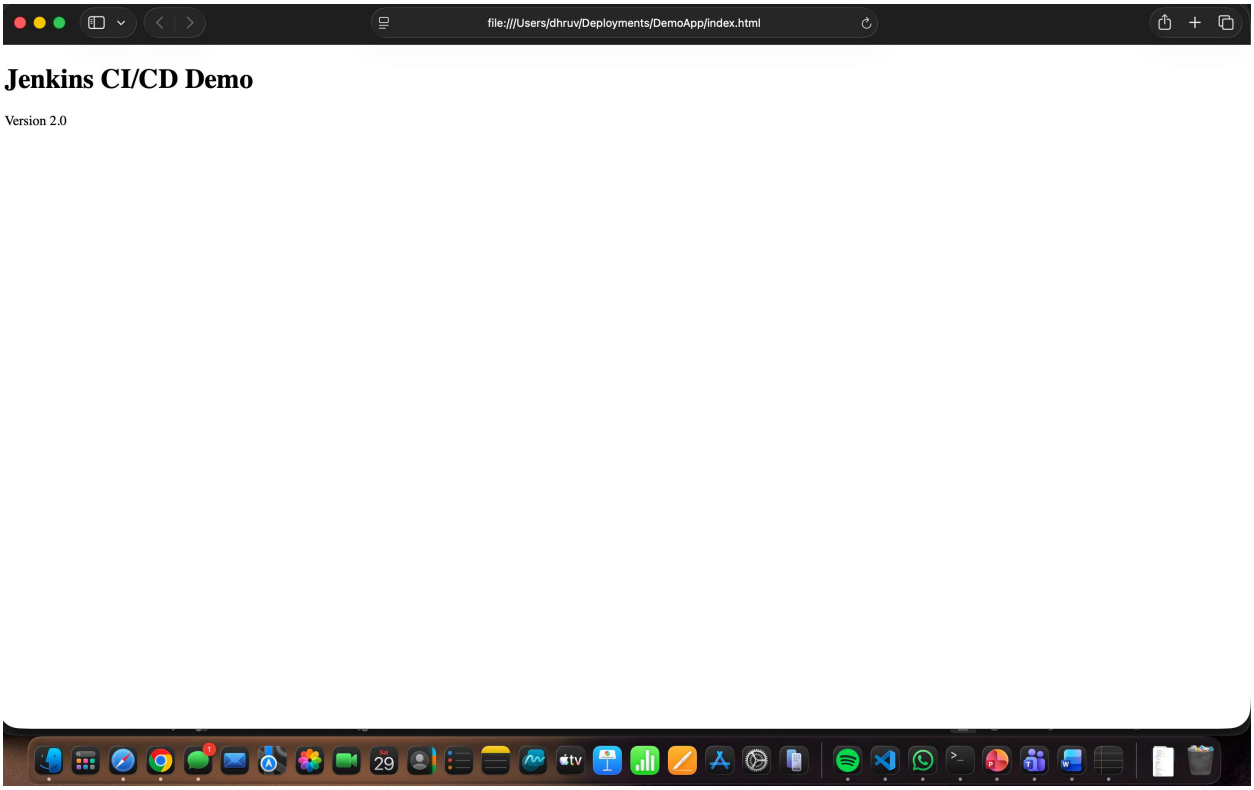
Step 7: CI/CD Update and Redeployment



```
Running on Jenkins in /Users/dhruv/.jenkins/workspace/GitHub-Jenkins-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
 > git rev-parse --resolve-git-dir /Users/dhruv/.jenkins/workspace/GitHub-Jenkins-Demo/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/dhruvh6/jenkins-cicd.git # timeout=10
Fetching upstream changes from https://github.com/dhruvh6/jenkins-cicd.git
 > git --version # timeout=10
 > git --version # 'git version 2.39.5 (Apple Git-154)'
 > git fetch --tags --force --progress -- https://github.com/dhruvh6/jenkins-cicd.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision a9f38192bdba11c5914d0fbe220cf39bf46bd8ac (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f a9f38192bdba11c5914d0fbe220cf39bf46bd8ac # timeout=10
Commit message: "Update version number from 1.0 to 2.0"
 > git rev-list --no-walk 938b2a276a56e25de2486d85cb2eb0b20342f43c # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
```



Step 8: Final Deployment Verification



CONCLUSION :

The experiment successfully demonstrated how GitHub and Jenkins can be combined to automate application deployment. By using a Declarative Jenkinsfile, the Build, Test, and Deploy tasks were organized into a repeatable pipeline, reducing the need for manual deployment.

The initial application as well as its modified version were successfully deployed through the pipeline. This showed that changes made in the GitHub repository could be automatically processed by Jenkins and delivered to the deployment environment. Overall, the experiment provided practical knowledge of CI/CD, Pipeline as Code, GitHub integration, and automated software deployment.